Secrets

Цели работы:

- изучить концепцию Secrets и их назначение в Kubernetes для безопасного хранения конфиденциальных данных.
- освоить способы создания Secrets: через командную строку и YAML-манифесты.
- научиться использовать Secrets в Pod: как переменные окружения и как файлы через Volumes.
- понять разницу между типами данных `data` (base64) и `stringdata` (plain-text).
- освоить базовые методы безопасной работы с Secrets.

`Secrets` — это объект Kubernetes, предназначенный для безопасного хранения конфиденциальной информации, такой как пароли, OAuth-токены, SSH-ключи. Использование Secrets позволяет отделить конфиденциальные данные от образов приложений и конфигурации Pod, уменьшая риск их утечки.

Пример 1: Создание Secret через командную строку и использование в качестве переменных окружения

Шаг 1: Создаем Secret с помощью `kubectl create secret generic`.

```
C:\Temp>kubectl create secret generic app-credentials --from-literal=DB_USERNAME=admin
--from-literal=DB_PASSWORD=super-secret-password
secret/app-credentials created

C:\Temp>
```

Пояснение: Команда создает Secret типа Opaque. Ключи и значения указываются напрямую.

Шаг 2: Создаем Pod, который использует данные из Secret как переменные окружения. (*YAML-манифест Pod, аналогичный примеру в ЛР7,* valueFrom и secretKeyRef используются для получения значений username и password из Secret my-secret).

app-secret-env-pod.yaml – Блокнот

Файл Правка Формат Вид Справка

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-secret-env-pod
spec:
  containers:
  - name: app-container
    image: nginx:alpine
    env:
      - name: DATABASE_USERNAME
        valueFrom:
          secretKeyRef:
            name: app-credentials      # Имя Secret, созданного в Шаге 1
            key: DB_USERNAME           # Ключ в Secret
            optional: false            # Если true - Pod запустится даже если ключа нет

      # Переменная DATABASE_PASSWORD получает значение из Secret
      - name: DATABASE_PASSWORD
        valueFrom:
          secretKeyRef:
            name: app-credentials      # Тот же Secret
            key: DB_PASSWORD           # Другой ключ в Secret
```

Стр 9, стлб 9 100% Windows (CRLF) UTF-8

Шаг 3: Проверяем, что переменные окружения установлены корректно.

```
C:\Temp>kubectl apply -f app-secret-env-pod.yaml
pod/app-secret-env-pod unchanged

C:\Temp>kubectl exec app-secret-env-pod -- env | findstr "DATABASE"
DATABASE_USERNAME=admin
DATABASE_PASSWORD=super-secret-password
```

***Примечание**: в Linux:* `kubectl exec app-secret-env-pod -- env | grep DATABASE`

Пример 2: Создание Secret через YAML-манифест и монтирование как файла

Шаг 1: Создаем YAML-манифест для Secret. В примере не реальный TLS сертификат, а просто тестовые данные. Реальный TLS сертификат можно получить, например, с помощью OpenSSL

tls-certificates.yaml – Блокнот

Файл Правка Формат Вид Справка

```
apiVersion: v1
kind: Secret
metadata:
  name: tls-certificates
type: Opaque
stringData:
  tls.crt: |
    -----BEGIN TEST CERTIFICATE-----
    This is a mock TLS certificate for demonstration purposes
    Used in Kubernetes Secrets example
    Not a real certificate - for testing only
    -----END TEST CERTIFICATE-----
  tls.key: |
    -----BEGIN TEST PRIVATE KEY-----
    This is a mock private key for demonstration purposes
    Used in Kubernetes Secrets example
    Not a real key - for testing only
    -----END TEST PRIVATE KEY-----
  app-config.properties: |
    database.host=postgresql-service
    database.port=5432
    api.timeout=30s
```

Примечание: Частичный текст `tls-certificates.yaml`

```
  tls.crt: |
    -----BEGIN TEST CERTIFICATE-----
    This is a mock TLS certificate for demonstration purposes
    Used in Kubernetes Secrets example
    Not a real certificate - for testing only
    -----END TEST CERTIFICATE-----
  tls.key: |
    -----BEGIN TEST PRIVATE KEY-----
    This is a mock private key for demonstration purposes
    Used in Kubernetes Secrets example
    Not a real key - for testing only
    -----END TEST PRIVATE KEY-----
  app-config.properties: |
    database.host=postgresql-service
    database.port=5432
    api.timeout=30s
```

Пояснение: Разница между data (требует base64) и stringData (автоматически кодируется в base64). Используем stringData для удобства записи многострочных конфигураций в чистом тексте.

Шаг 2: Создаем Pod, который монтирует этот Secret как файл в Volume (YAML-манифест Pod, где Secret app-secret монтируется в директорию /etc/app/secrets. Аналогично примеру с ConfigMap в ЛР7, но с secret вместо configMap) .

app-secret-volume-pod.yaml – Блокнот

Файл Правка Формат Вид Справка

```
apiVersion: v1
kind: Pod
metadata:
  name: app-secret-volume-pod
spec:
  containers:
  - name: app-container
    image: nginx:alpine
    volumeMounts:
    - name: secret-volume
      mountPath: /etc/app/secrets
      readOnly: true
  volumes:
  - name: secret-volume
    secret:
      secretName: tls-certificates
```

```
C:\Temp>kubectl apply -f tls-certificates.yaml
secret/tls-certificates created

C:\Temp>kubectl apply -f app-secret-volume-pod.yaml
pod/app-secret-volume-pod created
```

Шаг 3: Проверяем, что файл с секретом доступен внутри Pod.

Проверяем список файлов

```
C:\Temp>kubectl exec app-secret-volume-pod -- ls -la /etc/app/secrets/
total 4
drwxrwxrwt    3 root     root           140 Oct 30 16:01 .
drwxr-xr-x    3 root     root          4096 Oct 30 16:01 ..
drwxr-xr-x    2 root     root           100 Oct 30 16:01 ..2025_10_30_16_01_06.1601795818
lrwxrwxrwx    1 root     root            32 Oct 30 16:01 ..data -> ..2025_10_30_16_01_06.1601795818
lrwxrwxrwx    1 root     root            28 Oct 30 16:01 app-config.properties -> ..data/app-config.properties
lrwxrwxrwx    1 root     root            14 Oct 30 16:01 tls.crt -> ..data/tls.crt
lrwxrwxrwx    1 root     root            14 Oct 30 16:01 tls.key -> ..data/tls.key
```

Проверяем содержимое TLS сертификата

Проверяем содержимое конфигурации

```
C:\Temp>kubectl exec app-secret-volume-pod -- cat /etc/app/secrets/tls.crt
-----BEGIN TEST CERTIFICATE-----
This is a mock TLS certificate for demonstration purposes
Used in Kubernetes Secrets example
Not a real certificate - for testing only
-----END TEST CERTIFICATE-----

C:\Temp>kubectl exec app-secret-volume-pod -- cat /etc/app/secrets/app-config.properties
database.host=postgresql-service
database.port=5432
api.timeout=30s
```

Проверяем права доступа к файлам

```
C:\Temp>kubectl exec app-secret-volume-pod -- ls -l /etc/app/secrets/
total 0
lrwxrwxrwx    1 root     root            28 Oct 30 16:01 app-config.properties -> ..data/app-config.properties
lrwxrwxrwx    1 root     root            14 Oct 30 16:01 tls.crt -> ..data/tls.crt
lrwxrwxrwx    1 root     root            14 Oct 30 16:01 tls.key -> ..data/tls.key
```

Пример 3: Сравнение использования Secret и ConfigMap

Хотя Secrets и ConfigMap служат разным целям, их использование в Pod синтаксически очень похоже. Kubernetes предоставляет единый механизм для их интеграции, но с критически важными различиями в обработке и безопасности.

Сравнительная таблица: Secret vs ConfigMap

Критерий	ConfigMap	Secret
Назначение	Хранение нефункциональных, НЕконфиденциальных данных (номера портов, пути к файлам, флаги функциональности).	Хранение конфиденциальных данных (пароли, токены, ключи шифрования).
Синтаксис в Pod	Идентичен для обоих типов ресурсов. Используются секции configMapKeyRef и secretKey Ref в env.valueFrom, а также configMap и secret в volumes.	
Кодирование данных	Данные хранятся в чистом виде (plain text).	Данные хранятся в формате base64 (в поле data манифеста).
Безопасность в kubectl get	По умолчанию отображает все значения ключей в открытом виде.	По умолчанию НЕ показывает значения, заменяя их на <hidden> или *****.
Безопасность в etcd	Данные хранятся в незашифрованном виде (если не включено шифрование на уровне etcd).	Могут быть зашифрованы на уровне etcd (требуется настройка провайдера шифрования в кластере).
Рекомендуемый способ создания	kubectl create configmap ... или YAML-манифест с данными в чистом тексте.	kubectl create secret generic ... (автоматически кодирует в base64) или YAML-манифест с предварительно закодированными в base64 данными.

Практическая демонстрация различий

1. Создадим два объекта с одинаковыми данными:

Создаем ConfigMap

```
C:\Temp>kubectl create configmap example-config --from-literal=shared-key=my-super-secret-value
configmap/example-config created
```

Создаем Secret

```
C:\Temp>kubectl create secret generic example-secret --from-literal=shared-key=my-super-secret-value
secret/example-secret created
```

2. Посмотрим на разницу при получении объектов:

Проверяем ConfigMap. Вывод (данные видны):

```
C:\Temp>kubectl get configmap example-config -o yaml
apiVersion: v1
data:
  shared-key: my-super-secret-value
kind: ConfigMap
metadata:
  creationTimestamp: '2024-10-30T16:17:36Z"
  name: example-config
  namespace: default
  resourceVersion: "80513"
  uid: 65836e5d-7629-45c8-bca5-e24afc44df45
```

Проверяем Secret. Данные закодированы.

```
C:\Temp>kubectl get secret example-secret -o yaml
apiVersion: v1
data:
  shared-key: bXktc3VwZXItc2VjcmV0LXZhbHVl
kind: Secret
metadata:
  creationTimestamp: "2024-10-30T16:17:45Z"
  name: example-secret
  namespace: default
  resourceVersion: "80523"
  uid: 17b61830-6b88-4061-a398-8c0ca5d970ed
type: Opaque
```

3. Использование в Pod (синтаксис идентичен):
   В качестве переменных окружения:

```
# Для ConfigMap
env:
  - name: CONFIG_VALUE
    valueFrom:
      configMapKeyRef:
        name: example-config
        key: shared-key
# Для Secret
env:
  - name: SECRET_VALUE
```

```
    valueFrom:
      secretKeyRef:
        name: example-secret
        key: shared-key
```

В качестве файлов в Volume:

```
# Для ConfigMap
volumes:
  - name: config-volume
    configMap:
      name: example-config
# Для Secret
volumes:
  - name: secret-volume
    secret:
      secretName: example-secret
```

Вывод:

Kubernetes предоставляет единообразный API для работы с конфигурационными данными и секретами. Однако **Secret предназначен именно для конфиденциальной информации**, о чем свидетельствуют дополнительные меры защиты:

- автоматическое кодирование в base64.
- сокрытие содержимого при стандартных операциях kubectl get.
- возможность шифрования при хранении.

Важно: Base64 — это не шифрование, а способ кодирования. Данные можно легко декодировать. Поэтому Secrets следует рассматривать как *более безопасную альтернативу* хранению паролей в Pod-манифестах или ConfigMap, но для полноценной безопасности необходимо включать **шифрование Secrets на уровне etcd** и использовать строгие политики RBAC.

Задания для самостоятельного решения

Задание 8.1. Базовое использование Secrets

Создайте Secret с тремя ключами: api-key, auth-token, encryption-key
Создайте Pod, который использует:
- api-key как переменную API_KEY
- auth-token как переменную AUTH_TOKEN
- encryption-key как файл /app/secrets/encryption.key.

Проверочные команды:
```
kubectl get secret name-secret -o yaml
kubectl exec name-pod -- env | grep -E '(DB_USERNAME|DB_PASSWORD)'
```

Задание 8.2. Secret как файлы

Создайте Secret, содержащий два конфигурационных файла, и Pod, который монтирует эти файлы в разные директории с разными правами доступа.
Требования:
1. Secret с именем app-config-secret должен содержать:
   – ключ application.properties с многострочной конфигурацией приложения
   – ключ security.key с настройками безопасности
2. Pod с именем app-config-pod должен:
   – монтировать файл application.properties в директорию /etc/app/config/ с правами 0644
   – монтировать файл security.key в директорию /app/security/ с правами 0400 (только чтение)
   – использовать разные volumes для каждого пути монтирования

Проверочные команды:
```
kubectl get secret app-config-secret -o yaml

kubectl exec app-config-pod -- find /etc/app /app/security -type f | xargs ls -la

kubectl exec app-config-pod -- cat /etc/app/config/application.properties
kubectl exec app-config-pod -- cat /app/security/security.key

kubectl exec app-config-pod -- ls -l /etc/app/config/application.properties
kubectl exec app-config-pod -- ls -l /app/security/security.key
```

Задание 8.3. Комбинированное использование Secret и ConfigMap

Создайте ConfigMap app-config с ключом database.host (значение: postgresql-svc).
Создайте Secret app-secret с ключом database.password.
Создайте Pod, который:
– использует значение database.host из ConfigMap как переменную окружения DB_HOST.
– использует значение database.password из Secret как переменную окружения DB_PASSWORD.

Проверочные команды:

– Проверьте обе переменные окружения внутри Pod.

Задание 8.4. Диагностика и проблемы

Создайте Pod, который ссылается на несуществующий Secret. Изучите сообщения об ошибках.
Создайте Pod, который монтирует Secret, а затем удалите сам Secret. Что произойдет с Pod?

Проверочные команды:

```
kubectl describe pod <problem-pod-name>
kubectl get events --field-selector involvedObject.name=<problem-pod-name>
```

Контрольные вопросы

1. В чем основное отличие Secret от ConfigMap?
2. Какие два основных способа использования Secrets в Pod вы знаете?
3. Какой тип Secret используется по умолчанию при создании через kubectl create secret generic?
4. В чем разница между полями data и stringData в манифесте Secret?
5. Что произойдет с Pod, если он ссылается на несуществующий Secret?
6. Данные в Secret хранятся в зашифрованном виде в etcd. Правда или ложь? (Поясните ответ).
7. Как можно безопасно передать содержимое файла (например, сертификата) в Secret?
8. Какие команды вы бы использовали для диагностики проблем с Secrets?